# PyTorch 与神经网络：从张量到可训练模型

系统讲义 · 16 章 + 3 附录

2026 年 11 月

**摘　要**

本讲义共 16 章与 3 个附录，覆盖从张量与自动微分到卷积网络、序列模型、注意力机制，再到训练工程、性能优化与部署的完整链路。体例沿用既有讲义：每章分为定义、性质与定理、代码实现、算例、练习五部分；全部接口与行为以 PyTorch 官方文档为准，涉及版本差异处单独标注。附录 A 为 API 速查，附录 B 为练习要点，附录 C 为官方资源索引。

# 目录

读者定位 3
前置知识清单 3
阅读路线 4
第 1 章 张量：数据结构、设备与内存 5
第 2 章 张量运算、形状推理与广播 8
第 3 章 自动微分：Autograd 的机制与边界 11
第 4 章 最小可训练闭环：不用 `nn` 手写线性回归 14
第 5 章 `torch.nn`：Module、Parameter 与 state_dict 16
第 6 章 数据管道：Dataset、DataLoader 与 Transforms 19
第 7 章 损失函数与优化器 21
第 8 章 学习率调度与正则化 23
第 9 章 训练循环工程化：评估、检查点、日志与混合精度 25
第 10 章 卷积神经网络 28
第 11 章 序列模型：RNN、LSTM、GRU 30
第 12 章 注意力机制与 Transformer 32
第 13 章 GPU、显存与性能 34
第 14 章 保存、加载、导出与部署 36
第 15 章 调试、可复现性与常见错误 38
第 16 章 综合实战：MNIST → CIFAR-10 → 小型 NeRF 40
附录 A API 速查表 42
附录 B 练习参考答案要点 44
附录 C 官方资源索引 46
版本与勘误说明 47

本讲义面向**从零开始的读者**，目标不是”能跑通某一个项目”，而是**独立掌握用 PyTorch 设计、训练、调试、部署神经网络所需的全部核心知识**。全篇以官方文档（docs.pytorch.org）与官方教程（Learn the Basics）为准绳，凡与官方行为不一致之处会显式标注版本。

**关于版本口径**

- 以 **PyTorch 2.x**（2.9/2.11 实测）为准；涉及 1.x 差异时单独标注。
- 代码块均为可直接运行的最小片段，默认 `import torch`。
- 张量形状一律写成 `(B, C, H, W)` 形式，`B` 批大小、`C` 通道、`H`/`W` 高宽。
- 数学记号与代码变量一一对应；涉及本项目（CBF+NeRF）的实现时，标注文件与函数名。

---

## 读者定位

本讲义面向满足以下条件的读者：

- 具备 Python 基础：函数、类、列表/字典、`for`/`while`、`import`、异常处理；
- 具备高等数学与线性代数基础：导数、偏导、链式法则、矩阵乘法、转置；概率论知道期望与方差即可；
- **不要求**学过深度学习，不要求用过 NumPy（但用过会更快）；
- **不要求**有 GPU，全部示例可在 CPU 上跑通；
- 具备最基本的命令行能力：会用 `pip install`、能看懂 `python xxx.py`。

本讲义对每个术语先给一句定义再使用。

---

## 前置知识清单

| 编号 | 知识点 | 最低要求 | 出现章节 |
| --- | --- | --- | --- |
| P1 | 链式法则 | 会算 $\frac{d}{dx}f(g(x)) = f'(g(x))g'(x)$ | 第 3 章 |
| P2 | 矩阵乘法与转置 | 会算 $Ax$，知道 $(AB)^\top = B^\top A^\top$ | 第 2、4 章 |
| P3 | 偏导数与梯度 | 理解 $\nabla_\theta L$ 是与 $\theta$ 同形状的向量 | 第 3、4 章 |
| P4 | 概率基础 | 知道期望、方差、均匀/正态分布抽样 | 第 6、9 章 |
| P5 | Python 类与继承 | 会写 `class A: def __init__(self): ...` | 第 5 章 |
| P6 | Python 迭代器协议 | 知道 `__iter__` / `__len__` / `__getitem__` | 第 6 章 |
| P7 | 浮点表示 | 知道 float32 有 7 位有效十进制位，float16 更少 | 第 13、15 章 |
| P8 | 复数/卷积的直观 | 知道”滑窗加权求和”是什么 | 第 10 章 |

自测：能独立完成 P1、P2、P5 即可从第 1 章顺序阅读。

---

## 阅读路线

| 路线 | 目标读者 | 覆盖章节 | 预计用时 |
| --- | --- | --- | --- |
| A. 快速上手 | 要在一周内跑通并改一个训练脚本 | 第 1–9 章，每章只读 `### 定义`、`### 代码实现` | 8–12 小时 |
| B. 系统掌握 | 要独立设计模型、调参、排查训练异常 | 第 1–15 章全部小节，练习完成 2/3 | 40–60 小时 |
| C. 研究与工程 | 要做性能优化、分布式、部署 | 第 1–16 章 + 附录，另读官方《Performance Tuning Guide》《DDP Tutorial》 | 80 小时以上 |

推荐顺序就是章节编号顺序：张量 → 运算 → 自动微分 → 手写训练闭环 → `nn.Module` → 数据 → 损失与优化器 → 训练工程 → 卷积 → 序列 → 注意力 → GPU 与性能 → 保存部署 → 调试 → 综合实战。

---

# 第 1 章 张量：数据结构、设备与内存

PyTorch 的全部数据结构均以 `torch.Tensor` 为基础。本章界定张量的结构属性（形状、数据类型、设备）及其创建与转换方式，后续章节均以此为前提。

## 定义

**定义 1.1（张量）** 张量是**同构类型**元素组成的 $n$ 维数组，附带三个属性：形状（`shape`）、数据类型（`dtype`）、所在设备（`device`）。数学上 $A \in \mathbb{R}^{d_1 \times \cdots \times d_n}$。标量是 0 维张量，向量 1 维，矩阵 2 维，图像批 `(B,C,H,W)` 是 4 维。

```
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
x.shape, x.dtype, x.device
# (torch.Size([2, 2]), torch.float32, device(type='cpu'))
```

**定义 1.2（dtype）** 元素类型。深度学习常用：

| dtype | 位数 | 用途 | 备注 |
|---|---|---|---|
| `torch.float32` | 32 | **默认浮点**，训练主力 | 约 7 位十进制有效位 |
| `torch.float64` | 64 | 数值校验、科学计算 | 慢，GPU 上支持有限 |
| `torch.float16` / `bfloat16` | 16 | 混合精度训练 | 见第 9、13 章 |
| `torch.int64` | 64 | **索引、标签**（默认整型） | `torch.tensor([1,2])` 的默认 |
| `torch.bool` | 8 | 掩码 | `x > 0` 的结果 |

**定义 1.3（device）** 张量所在的物理/逻辑设备：`cpu`、`cuda:0`、`cuda:1`、`mps`（Apple）。**同一运算的所有张量必须在同一设备上**，否则报错。

**定义 1.4（叶张量 leaf tensor）** 由用户直接创建、`requires_grad=True`、且**不是**由其他张量运算得到的张量。只有叶张量的 `.grad` 会被累积。这个概念在第 3 章讲反向传播时是核心。

## 性质与定理

**性质 1.1（默认 dtype 规则）** `torch.tensor([1, 2, 3])` 是 `int64`；`torch.tensor([1., 2.])` 是 `float32`；`torch.zeros(3)` 是 `float32`。整数张量不能直接参与需要梯度的运算。

**性质 1.2（默认 device 规则）** 不带 `device=` 的构造一律在 CPU 上；`torch.zeros(3, device='cuda')` 直接在 GPU 上分配，比”先 CPU 再 `.cuda()`“少一次拷贝。

**性质 1.3（`torch.Tensor` 与 `torch.tensor`）** `torch.Tensor(...)` 是**类构造**，`torch.Tensor(3, 4)` 得到 $3\times4$ 的浮点张量，其内容为未初始化的内存值，不应假定其数值；`torch.tensor(data)` 是**工厂函数**，从数据推断 dtype。**应优先使用 `torch.tensor`、`torch.zeros`、`torch.empty` 等工厂函数。**

**性质 1.4（`.to()` 的语义）** `x.to(device)` 在设备不同时返回**新张量**并拷贝；设备相同时**返回自身**（不拷贝）。`x.cuda()` 等价于 `x.to('cuda')` 但不支持指定序号以外的参数。

**性质 1.5（视图与拷贝）** 下列操作**不拷贝数据**、与源共享内存：`view`、`reshape`（能共享时）、`permute`、`transpose`、`squeeze`、`unsqueeze`、切片、`expand`。下列操作**产生新数据**：`clone`、`contiguous`（必要时）、`repeat`、算术运算。共享存储意味着对其中一个张量的写入会反映在另一个张量上。

### 代码实现

```
import torch

# 1) 从数据构造
a = torch.tensor([1, 2, 3])                 # int64
b = torch.tensor([1.0, 2.0, 3.0])           # float32
c = torch.tensor([[1, 2], [3, 4]], dtype=torch.float64)

# 2) 形状工厂
torch.zeros(2, 3)          # 全 0
torch.ones(2, 3)           # 全 1
torch.full((2, 3), 7.0)    # 全 7
torch.empty(2, 3)          # 未初始化(不应假定数值)
torch.eye(3)               # 单位阵
torch.arange(0, 10, 2)     # [0,2,4,6,8]
torch.linspace(0, 1, 5)    # [0,.25,.5,.75,1]

# 3) 随机
torch.rand(3)              # U[0,1)
torch.randn(3)             # N(0,1)
torch.randint(0, 10, (3,)) # 整数
torch.manual_seed(42)      # 可复现(第 15 章详述)

# 4) 与 NumPy 互转(共享内存!)
import numpy as np
n = np.array([1.0, 2.0])
t = torch.from_numpy(n)    # 共享底层内存
n2 = t.numpy()             # 同样共享

# 5) dtype / device 转换
x = torch.randn(2, 3)
x.half()                   # float16
x.long()                   # int64
x.to('cuda' if torch.cuda.is_available() else 'cpu')

# 6) 形状操作(均不拷贝)
x = torch.randn(2, 3, 4)
x.view(6, 4)               # 要求内存连续
x.reshape(6, 4)            # 不连续时会自动拷贝
x.permute(2, 0, 1)         # 换维序 -> (4,2,3)
x.unsqueeze(0)             # (1,2,3,4)
x.squeeze(0)               # 去掉长度 1 的维度
x.expand(5, 2, 3, 4)       # 广播式扩张(不拷贝)
x.repeat(2, 1, 1)          # 真实复制

# 7) 连续性
y = x.permute(2, 0, 1)
y.is_contiguous()          # False
y.view(-1)                 # 抛出异常:张量不连续
y.reshape(-1)              # OK,会先拷贝
y.contiguous().view(-1)    # 显式做法
```

### 算例

**算例 1.1** 一张 RGB 图像，形状 `(3, 480, 640)`，要送入 CNN 需变成 `(1, 3, 480, 640)` 的批。用 `unsqueeze(0)`，不是 `view(1,3,480,640)`（后者在非连续时可能失败）。

**算例 1.2** 把 `(B, C, H, W)` 变成 `(B, H, W, C)`，用于可视化：`x.permute(0, 2, 3, 1)`。这一步**零拷贝**，但之后若要做 `.view()` 必须先 `.contiguous()`。

**算例 1.3** 计算显存占用：`(32, 3, 224, 224)` 的 float32 张量占 $32 \times 3 \times 224 \times 224 \times 4$ 字节 $= 19.3$ MB。反过来，若显卡 8 GB、模型参数 100 MB、中间激活是输入的 50 倍，就能判断批大小上限。

### 练习

1. 用 `torch.arange` 生成 0…99，`view(10,10)`，取第 3 行、取对角线、求每列和。
2. 构造两个张量验证”`view` 共享内存”：改原张量一个元素，看 view 是否变化。
3. 解释为什么 `torch.tensor([1.0, 2.0]).cuda()` 在小数据上比 `torch.tensor([...], device='cuda')` 慢。
4. 给定 `x = torch.randn(4, 3, 8, 8)`，写出得到 `(4, 8, 8, 3)` 且**不拷贝**的代码，并说明后续哪些操作会触发拷贝。
5. 一个 float16 张量与一个 float32 张量相加，结果 dtype 是什么？为什么？

---

# 第 2 章 张量运算、形状推理与广播

形状错误是 PyTorch 最高频的错误来源。本章把形状推理规则讲成可机械执行的步骤。

## 定义

**定义 2.1（逐元素运算 elementwise）** 两个**形状相同**的张量逐位置运算：加减乘除、`exp`、`log`、`sqrt`、`abs`、`clamp`、比较。结果形状不变。

**定义 2.2（广播 broadcasting）** 当两形状不同时，PyTorch 从**最右维**开始逐维对齐，每一维满足以下之一才能广播：1. 两维长度相等；2. 其中一维长度为 1（可被”拉伸”到另一维长度）；3. 其中一维不存在（视为长度 1）。

否则报错。广播**不复制数据**，是逻辑上的扩张（`expand` 的语义）。

**定义 2.3（归约 reduction）** 沿某一维或多个维聚合：`sum`、`mean`、`max`、`min`、`prod`、`argmax`、`argmin`、`norm`。用 `dim=` 指定维，`keepdim=True` 保留被归约的维（长度为 1）。

**定义 2.4（矩阵乘）** `torch.matmul` / `@` 支持批量矩阵乘：`(B, n, m) @ (B, m, p) -> (B, n, p)`。`torch.mm` 只接受 2 维；`torch.bmm` 只接受 3 维；`torch.einsum` 用下标语言表达任意收缩。

## 性质与定理

**定理 2.1（广播可行性判定）** 设 `A.shape = a`、`B.shape = b`，从右往左逐维比较。可行当且仅当对每个位置 $i$，$a_i = b_i$ 或 $a_i = 1$ 或 $b_i = 1$ 或该维只在一边出现。结果的第 $i$ 维长度为 $\max(a_i, b_i)$。

例：`(3,1,5)` 与 `(4,5)` 从右对齐：`5 vs 5` √；`1 vs 4` √（1 可拉伸）；`3 vs 缺` √。结果 `(3,4,5)`。

例：`(3,4)` 与 `(3,)` 从右对齐：`4 vs 3` ✗ → 报错。想按行相加应写 `(3,1)`。

**性质 2.1（原地运算）** 以 `_` 结尾的方法是原地操作：`x.add_(1)`。原地操作会**破坏自动微分**所需的历史值，若变量参与梯度计算会报 “a leaf Variable that requires grad is being used in an in-place operation”。

**性质 2.2（`dim` 的语义）** `dim=-1` 是最后一维，`dim=0` 是第 0 维。`x.sum(dim=0)` 消除第 0 维；`x.sum(dim=0, keepdim=True)` 保留为长度 1。

**性质 2.3（`max` 返回值）** `torch.max(x, dim=k)` 返回**命名元组** `(values, indices)`。只 `import torch` 时可直接解包；也可用 `x.max(dim=k).values`。这与 NumPy 的 `np.max` 不同。

**性质 2.4（einsum 的通用性）** `torch.einsum('bij,bjk->bik', A, B)` 等于批量矩阵乘；`torch.einsum('ij->ji', A)` 是转置；`torch.einsum('ii->', A)` 是迹。einsum 可读性好但通常略慢于专用算子。

## 代码实现

```
import torch

x = torch.randn(4, 3)
y = torch.randn(4, 3)

# 逐元素
x + y; x * y; x / (y.abs() + 1e-8)
x.exp(); x.log(); x.sqrt(); x.abs()
```

```
x.clamp(-1, 1)                # 截断到 [-1,1]
x.pow(2); x ** 2

# 归约
x.sum()                       # 全部求和(标量张量)
x.sum(dim=0)                  # 每列和 -> (3,)
x.mean(dim=0, keepdim=True)   # (1,3)
x.max(dim=1)                  # (values, indices)
x.argmax(dim=1)               # 只要索引

# 广播
a = torch.randn(3, 1, 5)
b = torch.randn(4, 5)
(a + b).shape                 # (3,4,5)

# 矩阵乘
A = torch.randn(2, 3); B = torch.randn(3, 4)
A @ B                         # (2,4)
batchA = torch.randn(8, 2, 3); batchB = torch.randn(8, 3, 4)
batchA @ batchB               # (8,2,4)

# einsum
torch.einsum('ij,jk->ik', A, B)
torch.einsum('b i j, b j k -> b i k', batchA, batchB)

# 拼接与拆分
torch.cat([A, A], dim=0)      # (4,3)
torch.stack([A, A], dim=0)    # (2,2,3) 新增一维
torch.split(A, 1, dim=0)      # 按长度切
torch.chunk(A, 2, dim=1)      # 均分

# 条件选择
mask = x > 0
torch.where(mask, x, torch.zeros_like(x))
x[mask]                       # 展平取出

# 排序
vals, idx = torch.sort(x, dim=0)
torch.topk(x, k=2, dim=1)
```

## 算例

**算例 2.1（批归一化的形状）** 输入 `(B, C, H, W)`，要对每个通道统计：

```
mean = x.mean(dim=(0, 2, 3), keepdim=True)   # (1,C,1,1)
var  = x.var(dim=(0, 2, 3), keepdim=True)    # (1,C,1,1)
x_norm = (x - mean) / (var + 1e-5).sqrt()    # 广播回 (B,C,H,W)
```

这里 `keepdim=True` 是关键——去掉它得到 `(C,)`，无法与 `(B,C,H,W)` 广播。

**算例 2.2（注意力分数的形状）** `Q:(B, h, N, d)`、`K:(B, h, M, d)`，分数 `scores = Q @ K.transpose(-2,-1)` 得 `(B,h,N,M)`；再 `scores.softmax(dim=-1)` 在”被关注的 M 个位置”上归一化。若将 `dim=-1` 写为 `dim=-2`，则归一化方向错误——这是注意力实现中的常见错误。

**算例 2.3（从零实现 softmax 并验证）**

```python
def my_softmax(x, dim=-1):
    x = x - x.max(dim=dim, keepdim=True).values   # 数值稳定
    e = x.exp()
    return e / e.sum(dim=dim, keepdim=True)
```

减去最大值不改变结果（分子分母同乘常数），但能避免 `exp` 溢出。PyTorch 的 `F.softmax` 内部同样这么做。

## 练习

1. 判断以下广播是否可行并写出结果形状：`(5,1)` 与 `(1,6)`；`(2,3)` 与 `(3,)`；`(2,1,3)` 与 `(4,3)`；`(1,1)` 与 `()`。
2. 用 `einsum` 实现：把 `(B,N,C)` 的每个样本转成 `(B,C,N)`。
3. 实现”对最后两维求 Frobenius 范数”，输出形状 `(B,)`。
4. 解释为什么 `x += 1` 对 `requires_grad=True` 的 x 可能报错，而 `x = x + 1` 不会。
5. 给定 `logits:(B,C)`，手写交叉熵（含 log-softmax），并与 `F.cross_entropy` 对比数值。

---

# 第 3 章 自动微分：Autograd 的机制与边界

PyTorch 的训练能力来自 autograd：你只写前向计算，反向梯度由框架自动求。本章讲清它**怎么工作、什么时候不工作**。

## 定义

**定义 3.1（计算图）** 前向运算时，autograd 记录每个运算的输入输出关系，形成一张**有向无环图**（DAG），边上是 `Function` 节点（如 `AddBackward0`）。叶节点是用户创建的张量，根节点是标量损失。

**定义 3.2（requires_grad）** 张量属性。为 `True` 时，所有由它参与产生的新张量都会带 `grad_fn`，并在 `backward()` 时累积梯度。`nn.Parameter` 默认为 `True`；普通张量默认 `False`。

**定义 3.3（叶张量）** 见定义 1.4。`x = torch.randn(3, requires_grad=True)` 是叶；`y = x * 2` 不是叶（`y.grad_fn` 非空）。**只有叶张量的 `.grad` 会被填充**；非叶张量的梯度需要 `y.retain_grad()` 才会保留。

**定义 3.4（反向传播）** `loss.backward()` 从 `loss` 出发沿图反向，对每个叶张量求 $\partial \text{loss}/\partial x$ 并**累加**到 `x.grad`。要求 `loss` 是标量；若 `loss` 是张量，必须传 `gradient=` 参数。

**定义 3.5（梯度累积）** 多次 `backward()` 不清零，梯度会**相加**。该行为是设计特性（可用于等效增大批大小），同时也是常见的实现错误来源。

## 性质与定理

**定理 3.1（链式法则的实现）** 设 $y = f(x)$、$L = g(y)$，反向时先算 $\partial L/\partial y$，再乘局部雅可比 $\partial y/\partial x$。对向量情形，autograd 用的是**向量-雅可比积**（VJP）：给定上游梯度 $v$，计算 $v^{\top} J$。因此一次 `backward()` 的代价与前向同阶。

**性质 3.1（图的释放）** 默认 `backward()` 结束后计算图被**释放**，再次 `backward()` 报 “Trying to backward through the graph a second time”。若需多次反传（如 GAN、二阶梯度），传 `retain_graph=True`，代价是显存不释放。

**性质 3.2（`torch.no_grad()`）** 上下文内不建图。用于**推理/评估**，可显著省显存。`torch.inference_mode()` 更激进（不记录版本计数），推理专用，比 `no_grad` 更快，但结果张量不能再参与 autograd。

**性质 3.3（`detach()`）** 返回与源共享内存、但从图中摘出的新张量。常用在：① 把张量当”常数”用；② 停止某条支路的梯度；③ 从带梯度的张量里取数据（`.detach().cpu().numpy()`）。

**性质 3.4（原地操作的危险）** 对需要梯度的张量做原地修改（`x.add_()`、`x[0]=1`、`x *= 2`）会破坏反向所需的值，可能报 “one of the variables needed for gradient computation has been modified by an inplace operation”。规则：**参与梯度的张量尽量不做原地写**。

**性质 3.5（自定义 Function）** 继承 `torch.autograd.Function`，实现 `forward` 与 `backward` 两个 `@staticmethod`，用 `ctx.save_for_backward` 保存中间量。这是写自定义算子的标准方式。

## 代码实现

```
import torch

# 1) 基本用法
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
```

```
y = (x ** 2).sum()          # y = 1 + 4 + 9 = 14
y.backward()
print(x.grad)               # d y/d x = 2x -> [2,4,6]

# 2) 梯度累积(常见错误来源)
y.backward()                # 再反传一次
print(x.grad)               # [4,8,12] —— 累加了!
x.grad.zero_()              # 标准写法:每步清零

# 3) 非叶张量
a = torch.tensor(2.0, requires_grad=True)
b = a * 3
c = b * b
c.backward()
print(a.grad)               # d(a*3)^2/da = 18a = 36
print(b.grad)               # None —— b 不是叶

# 4) retain_grad
b.retain_grad()
c = b * b
c.backward()
print(b.grad)               # 得 [12.]

# 5) no_grad / inference_mode
with torch.no_grad():
    z = x * 2               # z.requires_grad == False

# 6) detach
d = (x * 2).detach()        # 与 x 共享数据,但脱离图

# 7) 自定义 Function
class Square(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return x * x
    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        return grad_out * 2 * x

x = torch.tensor([3.0], requires_grad=True)
Square.apply(x).sum().backward()
print(x.grad)               # [6.]

# 8) 二阶梯度
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3
g, = torch.autograd.grad(y, x, create_graph=True)   # 3x^2
g2, = torch.autograd.grad(g, x)                      # 6x
print(g.item(), g2.item())  # 12.0 12.0
```

## 算例

**算例 3.1（为什么 loss 必须是标量）** `loss.backward()` 等价于对 `loss` 求导；若 `loss` 是 `(B,)` 向量，梯度 $J$ 是矩阵，无法自动选方向。实践上总写 `loss = criterion(pred, target)`，而 `criterion` 内部已经 `mean` / `sum`。

**算例 3.2（梯度校验）** 用有限差分验证 autograd 正确性：

```python
def grad_check(f, x, eps=1e-6):
    x = x.double().requires_grad_(True)
    y = f(x); y.backward()
    auto = x.grad.clone()
    num = torch.zeros_like(auto)
    for i in range(x.numel()):
        xp = x.detach().clone(); xp.view(-1)[i] += eps
        xm = x.detach().clone(); xm.view(-1)[i] -= eps
        num.view(-1)[i] = (f(xp) - f(xm)) / (2 * eps)
    return (auto - num).abs().max().item()
```

**算例 3.3（梯度累积模拟大 batch）** 显存只够 8 个样本，但想要 32 的等效批：

```python
optimizer.zero_grad()
for i, (x, y) in enumerate(loader):          # loader 批大小 8
    loss = criterion(model(x), y) / 4        # 除以累积步数
    loss.backward()
    if (i + 1) % 4 == 0:
        optimizer.step(); optimizer.zero_grad()
```

注意 `loss / 4` 与”最后再除”在数学上等价（梯度是线性的），但除以 4 能让中间梯度不溢出。

## 练习

1. 对 $f(x)=\log\sum_i e^{x_i}$ 手推 $\partial f/\partial x_i$（即 softmax），并用 autograd 验证。
2. 构造一个”原地操作破坏梯度”的最小例子，观察报错信息。
3. 用 `torch.autograd.grad` 求 $y=x^\top Ax$ 对 $x$ 的梯度，验证是 $(A+A^\top)x$。
4. 解释 `with torch.no_grad():` 与 `.detach()` 的区别：何时用哪个？
5. 写一个 `MyReLU` 自定义 Function，并在 $x=0$ 处讨论子梯度取 0 还是 1。

---

# 第 4 章 最小可训练闭环：不用 `nn` 手写线性回归

在引入 `nn.Module` 之前，先以纯张量与 autograd 完成一次完整的参数优化过程。该循环构成后续框架封装的语义基础。

## 定义

**定义 4.1（模型）** 从输入 $x$ 到输出 $\hat{y}$ 的参数化函数 $f_\theta$。线性回归取 $\hat{y} = Wx + b$，$\theta = \{W, b\}$。

**定义 4.2（损失函数）** 衡量预测与真值差距的标量函数 $L(\theta)$。回归常用均方误差 $\text{MSE} = \frac{1}{n}\sum(\hat{y}_i - y_i)^2$。

**定义 4.3（梯度下降）** 迭代更新 $\theta \leftarrow \theta - \eta \nabla_\theta L$，$\eta > 0$ 为学习率。小批量（mini-batch）版本用一批样本估计梯度。

**定义 4.4（epoch / step / batch）** 一个 **step** 是一次参数更新；一个 **epoch** 是把训练集完整过一遍；**batch size** 是每个 step 用的样本数。

## 性质与定理

**定理 4.1（收敛性，凸情形）** 若 $L$ 是 $\mu$-强凸且 $L$-光滑，取 $\eta \le 1/L$，则梯度下降满足 $L(\theta_k) - L^* \le (1 - \mu\eta)^k (L(\theta_0) - L^*)$，即线性收敛。深度学习不满足凸性，但小学习率下局部仍有类似的下降保证。

**性质 4.1（学习率的作用）** $\eta$ 太大会震荡甚至发散（越过极小值），太小会收敛极慢。经验起点：全连接/CNN 用 $10^{-3}$（Adam）或 $10^{-2}$（SGD），Transformer 常用 $10^{-4}$。

**性质 4.2（必须清零梯度）** PyTorch 的 `.grad` 是累积的；每个 step 前必须 `zero_grad()`，否则等价于把历史梯度一直加起来。

**性质 4.3（必须切换 train/eval）** 有 Dropout / BatchNorm 时，`model.train()` 与 `model.eval()` 行为不同（第 5、10 章）。手写版本没有这层，但引入 `nn` 后必须显式切换。

## 代码实现

```
import torch

torch.manual_seed(0)

# 造数据: y = 3x + 2 + 噪声
N = 200
x = torch.randn(N, 1)
true_w, true_b = 3.0, 2.0
y = true_w * x + true_b + 0.1 * torch.randn(N, 1)

# 参数(叶张量,需要梯度)
w = torch.randn(1, 1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)

lr = 0.1
batch = 32
for epoch in range(200):
    perm = torch.randperm(N)
    for i in range(0, N, batch):
        idx = perm[i:i + batch]
        xb, yb = x[idx], y[idx]
```

```
        pred = xb @ w + b                  # 前向
        loss = ((pred - yb) ** 2).mean()  # 损失

        loss.backward()                    # 反向

        with torch.no_grad():              # 更新时不需要建图
            w -= lr * w.grad
            b -= lr * b.grad
        w.grad.zero_()                     # 清零
        b.grad.zero_()

    if epoch % 50 == 0:
        print(f"epoch {epoch:3d}  loss {loss.item():.5f}  w {w.item():.3f}  b {b.item():.3f}")

# 期望 w ≈ 3, b ≈ 2
```

## 算例

**算例 4.1（为什么更新要放在 `no_grad` 里）** 参数更新 `w -= lr*w.grad` 本身是张量运算。若 `w` 需要梯度，PyTorch 会试图为”更新”再建图，既浪费内存又可能触发原地操作错误。标准写法是在 `torch.no_grad()` 内更新，或直接用优化器（优化器内部已处理）。

**算例 4.2（学习率过大直接发散）** 把上面 `lr` 改成 1.0，你会看到 loss 变成 `nan`。原因：$\eta > 1/L$ 时迭代不收敛，梯度爆炸后数值溢出。排查顺序见第 15 章。

**算例 4.3（手动实现动量）**

```
velocity = torch.zeros_like(w)
velocity = 0.9 * velocity + w.grad          # 累积方向
w -= lr * velocity
```

这就是 SGD with momentum，官方 `torch.optim.SGD(momentum=0.9)` 的内部逻辑。

## 练习

1. 把模型改成 $y = w_2 x^2 + w_1 x + b$，用同一套循环拟合。
2. 实现”每 20 个 epoch 学习率减半”，观察收敛曲线。
3. 用 `torch.autograd.grad` 代替 `loss.backward()` 做更新，比较两种写法。
4. 加入 L2 正则 $\lambda \,/\!/\, w \,/\!/^2$，解释它等价于 weight decay 的条件（见第 8 章）。
5. 故意不做 `zero_grad`，画出 loss 曲线并解释现象。

---

# 第 5 章 `torch.nn`：Module、Parameter 与 state_dict

第 4 章的参数是手工维护的裸张量。实践中模型由多层复合而成，需要统一的参数注册、设备迁移、序列化与模式切换机制。`torch.nn` 提供的正是这套机制。

## 定义

**定义 5.1（Parameter）** `nn.Parameter` 是 `Tensor` 的子类，构造时自动 `requires_grad=True`，并被 `nn.Module` 识别为”需要训练的参数”。只有被注册的 `Parameter` 才会出现在 `module.parameters()` 与 `state_dict()` 中。

**定义 5.2（Module）** `nn.Module` 是模型的基类。其核心职责有三：① 通过属性赋值**自动注册**子模块与参数；② 提供 `forward` 的调用入口（`__call__` 中插入 hook 与模式处理）；③ 递归遍历子模块。自定义模型必须继承它并实现 `forward`。

**定义 5.3（buffer）** 需要随模型保存、但不参与梯度更新的张量（如 BatchNorm 的滑动均值）。通过 `register_buffer(name, tensor)` 注册；`persistent=False` 时不被写入 `state_dict`。

**定义 5.4（state_dict）** 从参数名到张量的有序字典，反映模块树的层级结构（如 `"encoder.0.weight"`）。它是**唯一推荐的保存格式**。

**定义 5.5（hook）** 在模块前向/反向的特定时刻插入回调的机制：`register_forward_hook`、`register_forward_pre_hook`、`register_full_backward_hook`。用于特征提取、可视化与调试，不改变模块定义。

## 性质与定理

**性质 5.1（注册机制）** 对 `Module` 实例执行 `self.x = nn.Linear(...)` 或 `self.p = nn.Parameter(...)` 时，`__setattr__` 会把它们登记到 `_modules` / `_parameters` 中。若把同类型对象放进普通 Python 列表（`self.layers = [nn.Linear(...), ...]`），则**不会注册**，优化器将看不到这些参数；正确写法是 `nn.ModuleList` 或 `nn.Sequential`。

**性质 5.2（`__call__` 与 `forward`）** 调用 `model(x)` 实际执行 `Module.__call__`，其中依次运行前置 hook、`forward`、后置 hook。**应重写 `forward`，不应重写 `__call__`**，否则 hook 机制失效。

**性质 5.3（train/eval 的传播）** `model.train()` 与 `model.eval()` 仅递归设置 `self.training` 标志，不改变参数数值。受影响的是那些在前向中读取该标志的层：`Dropout`（训练时随机置零）、`BatchNorm`（训练时用批统计并更新滑动均值，评估时用滑动均值）。**推理前忘记 `eval()` 会导致结果随批内容波动**。

**性质 5.4（`state_dict` 与设备无关）** `state_dict()` 返回的张量仍在原设备上，但在 `torch.save` 时被序列化为 CPU 可存储格式；加载时用 `map_location` 指定目标设备，可跨 CPU/GPU 迁移。

**性质 5.5（参数初始化）** 默认初始化由各层决定（`Linear` 用 $U(-\sqrt{k}, \sqrt{k})$，$k = 1/\text{fan_in}$）。自定义初始化须在 `reset_parameters()` 或 `__init__` 末尾完成，并使用 `torch.no_grad()` 包裹，避免被 autograd 记录。

## 代码实现

```
import torch
import torch.nn as nn

# 1) 最小自定义模型
class MLP(nn.Module):
    def __init__(self, in_dim, hidden, out_dim, p=0.1):
        super().__init__()                          # 必须调用
        self.fc1 = nn.Linear(in_dim, hidden)
        self.act = nn.ReLU()
        self.drop = nn.Dropout(p)
        self.fc2 = nn.Linear(hidden, out_dim)

    def forward(self, x):                           # x: (B, in_dim)
        x = self.fc1(x)
        x = self.act(x)
        x = self.drop(x)
        return self.fc2(x)                          # (B, out_dim)

model = MLP(784, 256, 10)
print(sum(p.numel() for p in model.parameters()))     # 参数量
print([n for n, _ in model.named_parameters()])
print(model.state_dict().keys())

# 2) 容器
seq = nn.Sequential(nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 1))
layers = nn.ModuleList([nn.Linear(10, 10) for _ in range(3)])
layers_dict = nn.ModuleDict({"a": nn.Linear(10, 5)})

# 3) 参数与 buffer
class Norm(nn.Module):
    def __init__(self):
        super().__init__()
        self.scale = nn.Parameter(torch.ones(1))
        self.register_buffer("running_mean", torch.zeros(1))

# 4) 设备迁移与 dtype
model.to("cuda")
model.to(torch.float16)

# 5) 参数遍历/冻结
for name, p in model.named_parameters():
    if name.startswith("fc1"):
        p.requires_grad_(False)          # 冻结

# 6) 保存/加载(见第 14 章详述)
torch.save(model.state_dict(), "m.pt")
model2 = MLP(784, 256, 10)
model2.load_state_dict(torch.load("m.pt", map_location="cpu", weights_only=True))

# 7) hook 取中间特征
feats = {}
def hook(module, inp, out):
    feats["fc1"] = out.detach()
model.fc1.register_forward_hook(hook)

# 8) 初始化
def init_weights(m):
```

```
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity="relu")
        nn.init.zeros_(m.bias)
model.apply(init_weights)
```

### 算例

**算例 5.1（参数量计算）** 一个 `Linear(in, out)` 含 `out*in` 个权重与 `out` 个偏置，共 `out*(in+1)`。两层 MLP `784→256→10`：$256\times 785 + 10\times 257 = 200,960 + 2,570 = 203,530$。这是估算显存与计算量的第一步。

**算例 5.2（误用普通列表导致参数不更新）**

```
class Bad(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = [nn.Linear(10, 10) for _ in range(2)]   # 未注册
```

此时 `model.parameters()` 为空，优化器拿不到任何参数。改为 `nn.ModuleList` 即可。

**算例 5.3（BatchNorm 的 train/eval 差异）**

```
bn = nn.BatchNorm1d(4)
bn.train()
bn(torch.randn(8, 4))                    # 用批统计,更新 running_mean
bn.eval()
bn(torch.randn(8, 4))                    # 用 running_mean,不更新
```

若推理时未切 `eval()`，同一输入在不同批中输出不同，模型不可复现。

### 练习

1. 实现一个两层 MLP，并用 `named_parameters()` 打印每个参数的形状。
2. 解释 `nn.Sequential` 与 `nn.ModuleList` 的适用场景差异。
3. 给模型加一个 `register_buffer`，验证它出现在 `state_dict` 中而**不**出现在 `parameters()` 中。
4. 用 `register_forward_hook` 提取中间层特征并计算其均值，用于判断激活是否饱和。
5. 冻结 `fc1` 后训练若干步，验证 `fc1.weight.grad` 为 `None`。

---

# 第 6 章 数据管道：Dataset、DataLoader 与 Transforms

模型的前向与反向只是训练的一部分；数据供给的效率与正确性同样决定实验成败。

## 定义

**定义 6.1（Dataset）** 表示一个样本集合的对象。实现 `__len__` 与 `__getitem__` 即为”映射式”数据集（`torch.utils.data.Dataset`）；实现 `__iter__` 则可用于流式数据（`IterableDataset`）。

**定义 6.2（DataLoader）** 把 Dataset 包装为可迭代的批量加载器：负责采样（`Sampler`）、批内拼接（`collate_fn`）、并行加载（`num_workers`）与内存布局（`pin_memory`）。

**定义 6.3（collate_fn）** 把若干单样本合并为一个批的函数。默认实现对张量做 `torch.stack`，对定长序列同样有效；样本长度可变时必须自定义。

**定义 6.4（Sampler）** 决定样本顺序的迭代器。`RandomSampler`（默认，`shuffle=True`）、`SequentialSampler`、`WeightedRandomSampler`（类别不均衡）、`DistributedSampler`（多卡）。

**定义 6.5（Transform）** 应用于单个样本的预处理/增强函数，典型用法是 `torchvision.transforms` 的组合。

## 性质与定理

**性质 6.1（索引语义）** `dataset[i]` 必须返回可被 `collate_fn` 合并的单样本；典型返回 `(x, y)`，`x` 为 `(C,H,W)` 张量，`y` 为标量或定长向量。批量形状由 `collate_fn` 决定，通常为 `(B,C,H,W)`。

**性质 6.2（`num_workers` 与进程）** `num_workers>0` 时，每个 worker 是独立进程，通过 `fork`（Linux 默认）或 `spawn` 创建。数据在 worker 中反序列化后经共享内存传给主进程。`num_workers=0` 表示在主进程内同步加载。经验取值：`min(4, CPU 核数)`；过多会导致内存与上下文切换开销反超收益。

**性质 6.3（迭代顺序）** 每次 `for batch in loader:` 都会重新创建迭代器，因此 `shuffle=True` 时每个 epoch 的顺序不同。但若在循环外手动 `iter(loader)` 并复用，则顺序固定。

**性质 6.4（`drop_last`）** 训练时通常 `drop_last=True`，避免最后一个不完整批对 BatchNorm 统计的扰动；评估时应为 `False`，以免漏算样本。

**性质 6.5（`pin_memory` 与 `non_blocking`）** 设为 `True` 时，批数据被放入锁页内存，配合 `tensor.to(device, non_blocking=True)` 可实现主机到设备传输与计算重叠。仅在 GPU 训练时有效。

## 代码实现

```
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

class TabularDataset(Dataset):
    def __init__(self, X, y, transform=None):
        self.X, self.y, self.transform = X, y, transform

    def __len__(self):
        return len(self.X)
```

```
    def __getitem__(self, i):
        x = self.X[i]
        if self.transform:
            x = self.transform(x)
        return x, self.y[i]

ds = TabularDataset(torch.randn(1000, 784), torch.randint(0, 10, (1000,)))
loader = DataLoader(ds, batch_size=64, shuffle=True,
                    num_workers=4, pin_memory=True, drop_last=True)

for xb, yb in loader:
    print(xb.shape, yb.shape)          # (64,784) (64,)
    break

# 图像增强
img_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),                        # -> (C,H,W), float32, [0,1]
    transforms.Normalize(mean=[0.485,0.456,0.406],
                         std=[0.229,0.224,0.225]),
])

# 变长序列: 自定义 collate
def pad_collate(batch):
    xs, ys = zip(*batch)
    xs = torch.nn.utils.rnn.pad_sequence(xs, batch_first=True)
    return xs, torch.stack(ys)

# 类别不均衡
from torch.utils.data import WeightedRandomSampler
weights = torch.tensor([1.0 / c for c in counts])   # 逆频率
sampler = WeightedRandomSampler(weights, num_samples=len(ds), replacement=True)
```

## 算例

**算例 6.1（一个 epoch 的批数与步数）** 数据集 $N=1000$、`batch_size=64`、`drop_last=True`，则每 epoch 步数 $\lfloor 1000/64 \rfloor = 15$。若总训练 10 个 epoch，则共 150 次参数更新。

**算例 6.2（worker 数与吞吐）** 设单批计算耗时 $t_c$、单批加载耗时 $t_d$。`num_workers=k` 时流水线吞吐近似 $\frac{1}{\max(t_c,\ t_d/k)}$。当 $t_d/k < t_c$ 后继续增加 worker 不再提升吞吐，只会增加内存占用。

**算例 6.3（`IterableDataset` 与多 worker 的重复读）** 流式数据集若按文件切分给 worker，必须用 `torch.utils.data.get_worker_info()` 让每个 worker 只读自己那份，否则会出现样本重复。

## 练习

1. 实现一个把图像随机裁剪为 $32\times32$ 的 Dataset，并打印一个批的形状。
2. 比较 `num_workers=0` 与 `num_workers=4` 在一个 epoch 上的实际耗时。
3. 写一个 `collate_fn` 支持每条样本长度不同的序列，并说明 padding 值的选取原则。
4. 用 `WeightedRandomSampler` 使二分类的 1:9 数据在每个批中接近 1:1。
5. 解释为什么在 `num_workers>0` 时不宜在 Dataset 中持有可能被并发修改的状态。

---

# 第 7 章 损失函数与优化器

## 定义

**定义 7.1（损失函数）** 把模型输出与目标映射为标量的可微函数。按任务分为回归（`MSELoss`、`L1Loss`、`HuberLoss`）、分类（`CrossEntropyLoss`、`BCEWithLogitsLoss`）、排序/度量（`TripletMarginLoss`、`CosineEmbeddingLoss`）。

**定义 7.2（优化器）** 依据梯度更新参数的算法对象。`torch.optim` 中的优化器持有 `param_groups`，每组的超参数（`lr`、`weight_decay`）可独立设置。

**定义 7.3（参数组）** 形如 `[{"params": [...], "lr": 1e-3}, {"params": [...], "lr": 1e-2}]` 的列表，用于对不同部分使用不同超参数（如微调时冻结主干、放大头部学习率）。

## 性质与定理

**性质 7.1（`CrossEntropyLoss` 的输入约定）** 输入为**未归一化的 logits**，形状 `(B,C)` 或 `(B,C,d1,d2,...)`；目标为 `int64` 类别索引，形状 `(B,)` 或 `(B,d1,...)`。该函数内部完成 `log_softmax` 与负对数似然，因此**前向不应再接 `softmax`**。若已 softmax，应改用 `NLLLoss` 并取对数。

**性质 7.2（`BCEWithLogitsLoss` 的输入约定）** 输入为 logits，目标为同形状的浮点张量（取值 0/1）。它内部做 sigmoid 与 BCE 的合并，数值上比”先 sigmoid 再 BCE”更稳定。

**性质 7.3（优化器与梯度的关系）** 优化器只读取 `.grad`，不参与前向。`optimizer.step()` 之后必须 `zero_grad()`（或在下一步之前清零）；某些实现把 `zero_grad(set_to_none=True)` 作为默认，可略微节省内存。

**性质 7.4（SGD 与 Adam 的更新式）** SGD：$\theta \leftarrow \theta - \eta g$。Momentum：$v \leftarrow \beta v + g$，$\theta \leftarrow \theta - \eta v$。Adam：$m \leftarrow \beta_1 m + (1-\beta_1) g$，$v \leftarrow \beta_2 v + (1-\beta_2) g^2$，$\hat{m} = \frac{m}{1-\beta_1^t}$，$\hat{v} = \frac{v}{1-\beta_2^t}$，$\theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}}+\epsilon}$。AdamW：在 Adam 基础上把权重衰减与梯度更新解耦，$\theta \leftarrow \theta - \eta(\frac{\hat{m}}{\sqrt{\hat{v}}+\epsilon} + \lambda\theta)$。

**性质 7.5（weight decay 与 L2 正则的等价性）** 对**纯 SGD**，`weight_decay=λ` 与在损失中加 $\frac{\lambda}{2}\,/\!/\,\theta\,/\!/^2$ 等价；对 **Adam**，二者不等价，后者会被自适应学习率缩放。若需要真正的 L2 正则，应使用 AdamW 或显式在损失中加入惩罚项。

## 代码实现

```
import torch
import torch.nn as nn

# 分类
crit = nn.CrossEntropyLoss(label_smoothing=0.1)
logits = torch.randn(8, 10)          # 未过 softmax
target = torch.randint(0, 10, (8,))
loss = crit(logits, target)

# 二分类
crit_b = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([3.0]))
logit_b = torch.randn(8, 1)
loss_b = crit_b(logit_b, torch.randint(0, 2, (8, 1)).float())

# 回归
```

```python
crit_r = nn.MSELoss()
# 优化器
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)

# 分参数组：主干小学习率，头部大学习率
opt = torch.optim.SGD([
    {"params": model.backbone.parameters(), "lr": 1e-4},
    {"params": model.head.parameters(),     "lr": 1e-2},
], momentum=0.9)

# 梯度裁剪（循环网络与大模型常用）
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

## 算例

**算例 7.1（logits 与 softmax 误用）** 若模型末尾已经 `softmax`，再送 `CrossEntropyLoss`，相当于对概率取 `log_softmax`，梯度被压缩，训练显著变慢且不收敛。判断方法：检查 `model.forward` 的最后一行是否含 `softmax`/`sigmoid`。

**算例 7.2（类别不均衡）** 二分类中正样本占 5%。此时 `BCEWithLogitsLoss` 的梯度被负样本主导，模型倾向全预测为负。手段：`pos_weight` 加权、重采样（`WeightedRandomSampler`）、或改用 Focal Loss。

**算例 7.3（AdamW 与 Adam 的差异）** 在 Transformer 类模型上，AdamW 通常比 Adam 泛化更好，因为解耦后的权重衰减不会被 $1/\sqrt{\hat{v}}$ 放大。实践默认：Transformer 用 AdamW，CNN 用 SGD+Momentum 或 AdamW。

## 练习

1. 手写 `cross_entropy` 并与 `nn.CrossEntropyLoss` 对比数值。
2. 构造一个类别极不均衡的数据集，分别用 `pos_weight` 与重采样改善召回率。
3. 推导 Adam 的偏差修正项，说明 $1-\beta^t$ 的来源。
4. 比较同一模型在 SGD、SGD+Momentum、AdamW 下的收敛曲线。
5. 解释为什么 `zero_grad(set_to_none=True)` 能节省内存。

---

# 第 8 章 学习率调度与正则化

## 定义

**定义 8.1（学习率调度器）** 按训练进度调整优化器学习率的对象，位于 `torch.optim.lr_scheduler`。调用顺序固定为：`optimizer.step()` → `scheduler.step()`。

**定义 8.2（常用调度策略）** - `StepLR`：每 $T$ 个 epoch 乘以 $\gamma$； - `MultiStepLR`：在指定 epoch 乘以 $\gamma$； - `CosineAnnealingLR`：$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})(1 + \cos\frac{t\pi}{T})$； - `OneCycleLR`：前 $p$ 比例线性升温到峰值，再余弦退火； - `ReduceLROnPlateau`：监控指标不再改善时衰减。

**定义 8.3（正则化）** 任何限制模型复杂度、降低泛化误差的手段，包括权重衰减、Dropout、数据增强、早停、标签平滑。

**定义 8.4（Dropout）** 训练时以概率 $p$ 将神经元输出置零，并把保留值放大 $1/(1-p)$（inverted dropout），使期望不变；评估时不做任何操作。

**定义 8.5（早停）** 监控验证集指标，连续 $k$ 个 epoch 未改善则终止训练并回滚到最优检查点。

## 性质与定理

**性质 8.1（学习率与批大小的关系）** 线性缩放经验法则：批大小乘以 $k$ 时，学习率乘以 $k$（或按 $\sqrt{k}$ 缩放，视优化器而定）。依据是小批量梯度的方差约为 $1/B$，增大 $B$ 使梯度更平稳，可承受更大步长。

**性质 8.2（Dropout 与 BatchNorm 的相互作用）** 二者同时使用时，Dropout 会改变批统计，导致 BatchNorm 的方差估计偏大。常见做法是仅在 BatchNorm 之后、且 $p$ 取较小值（0.1 以下）时使用，或改用其他正则化手段。

**性质 8.3（权重衰减的层选择性）** 通常不对偏置项与归一化层的仿射参数施加权重衰减；对卷积与线性层权重施加。实现上通过参数组区分。

**性质 8.4（标签平滑）** 把 one-hot 目标 $\delta_c$ 替换为 $(1-\epsilon)\delta_c + \epsilon/C$，等价于在损失中加入对均匀分布的 KL 惩罚，可缓解过置信。

## 代码实现

```
import torch
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR, ReduceLROnPlateau

opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
sched = CosineAnnealingLR(opt, T_max=50, eta_min=1e-6)

for epoch in range(50):
    train_one_epoch()
    validate()
    opt.step() if False else None     # 优化器 step 在批循环内
    sched.step()                      # 每个 epoch 调一次

# 按批调度
sched_batch = OneCycleLR(opt, max_lr=1e-3, total_steps=len(loader)*epochs)

# 按指标调度
sched_plateau = ReduceLROnPlateau(opt, mode="min", factor=0.5, patience=3)

# 分组权重衰减：偏置与归一化参数不衰减
```

```python
decay, no_decay = [], []
for n, p in model.named_parameters():
    (no_decay if p.ndim <= 1 or "bn" in n else decay).append(p)
opt = torch.optim.AdamW([
    {"params": decay,    "weight_decay": 1e-2},
    {"params": no_decay, "weight_decay": 0.0},
], lr=3e-4)
```

## 算例

**算例 8.1（调度器调用次数错误）** 若把按 epoch 设计的 `CosineAnnealingLR(T_max=50)` 在每个**批**后调用，则 50 步之后学习率就退到下限，训练提前停滞。判断依据：调度器的 `T_max` 单位必须与 `step()` 的调用频率一致。

**算例 8.2（OneCycleLR 的收益）** 在固定总步数下，先升温再退火通常比恒定学习率高 1–2 个百分点的准确率，且训练更稳定。代价是需要预先知道 `total_steps`。

**算例 8.3（早停与学习率衰减的配合）** 先以 `ReduceLROnPlateau` 衰减到很小，再以早停终止，通常比固定 epoch 更节省算力，并能避免末期过拟合。

## 练习

1. 在同一任务上比较恒定学习率与余弦退火的验证集曲线。
2. 实现一个自定义调度器（每 $k$ 步线性升温），继承 `LambdaLR`。
3. 构造参数组，使偏置与 BatchNorm 参数不参与权重衰减。
4. 讨论 Dropout 概率 $p$ 与网络宽度的关系：宽网络是否需要更大的 $p$？
5. 实现标签平滑的两种写法（`CrossEntropyLoss(label_smoothing=...)` 与手动混合），验证数值一致。

---

# 第 9 章 训练循环工程化：评估、检查点、日志与混合精度

## 定义

**定义 9.1（训练循环）** 由若干 epoch 组成；每个 epoch 依次执行训练阶段与验证阶段；每个批执行”前向—损失—反向—更新”。

**定义 9.2（检查点 checkpoint）** 保存训练状态的产物，至少包含 `model_state_dict`、`optimizer_state_dict`、`epoch`、`best_metric`；完整恢复还需 `scheduler_state_dict` 与随机数状态。

**定义 9.3（混合精度训练 AMP）** 前向与反向部分使用 `float16`/`bfloat16`，同时保留 `float32` 的主权重副本，以降低显存占用并提升吞吐。

**定义 9.4（梯度缩放）** 由于 `float16` 的最小正规数约为 $6\times10^{-8}$，小梯度会下溢为 0。`GradScaler` 先把损失放大 $2^k$，反向后再除回，并在检测到 `inf`/`nan` 时跳过该步并降低 $k$。

## 性质与定理

**性质 9.1（评估必须 `no_grad` + `eval`）** 评估时应在 `torch.no_grad()`（或 `inference_mode()`）下前向，并调用 `model.eval()`。前者省显存与计算，后者修正 Dropout 与 BatchNorm 的行为。二者缺一都会使指标失真。

**性质 9.2（检查点保存的内容与体积）** 仅保存 `state_dict` 时体积约为参数量的 4 字节/参数（float32）；若用 `torch.save(model)` 保存整个对象，体积更大且与代码结构强耦合，不推荐。恢复训练必须同时保存优化器状态，否则动量与二阶矩丢失，学习率调度的连续性也会中断。

**性质 9.3（AMP 的适用范围）** 矩阵乘、卷积等适合半精度；`softmax`、`log`、`sum`、归一化等易溢出或损失精度的算子应保留 `float32`。`torch.autocast` 会自动选择合适精度，无需手工转换。

**性质 9.4（AMP 与自定义算子）** 自定义 `Function` 若在半精度下不稳定，可在 `forward` 内显式 `x.float()`，返回时再转回。

## 代码实现

```
import torch
from torch.amp import autocast, GradScaler

scaler = GradScaler("cuda") if torch.cuda.is_available() else None

def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total, correct, loss_sum = 0, 0, 0.0
    for x, y in loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        if scaler is not None:
            with autocast("cuda", dtype=torch.float16):
                logits = model(x)
                loss = criterion(logits, y)
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(optimizer)
            scaler.update()
```

```
        else:
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
        loss_sum += loss.item() * y.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        total += y.size(0)
    return loss_sum / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total, correct = 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        correct += (logits.argmax(1) == y).sum().item()
        total += y.size(0)
    return correct / total

def save_ckpt(path, model, optimizer, scheduler, epoch, best):
    torch.save({
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(),
        "epoch": epoch,
        "best": best,
    }, path)

def load_ckpt(path, model, optimizer=None, scheduler=None):
    ckpt = torch.load(path, map_location="cpu", weights_only=False)
    model.load_state_dict(ckpt["model"])
    if optimizer is not None and "optimizer" in ckpt:
        optimizer.load_state_dict(ckpt["optimizer"])
    if scheduler is not None and "scheduler" in ckpt:
        scheduler.load_state_dict(ckpt["scheduler"])
    return ckpt.get("epoch", 0), ckpt.get("best", float("inf"))
```

## 算例

**算例 9.1（AMP 的显存与速度收益）** 在支持 Tensor Core 的 GPU 上，`float16` 矩阵乘的理论吞吐可达 `float32` 的数倍；显存占用约减半（激活与权重为 16 位，主权重仍为 32 位）。实际端到端加速通常为 1.3–2 倍，受限于访存与数据加载。

**算例 9.2（`scaler.unscale_` 与梯度裁剪）** 若直接对缩放后的梯度做裁剪，`max_norm` 的语义被 $2^k$ 放大，裁剪失效。正确顺序是 `scale(loss).backward()` → `unscale_(optimizer)` → `clip_grad_norm_` → `scaler.step()`。

**算例 9.3（续训的等价性）** 完整恢复需要模型、优化器、调度器三者状态与 epoch 计数。缺优化器状态会导致续训初期 loss 突增（动量被清零）；缺调度器状态会导致学习率回到初值。

## 练习

1. 实现一个 `Trainer` 类，封装训练、验证、检查点与早停。
2. 对比开启/关闭 AMP 时的显存峰值与每 epoch 耗时。
3. 在中途中断并续训，验证 loss 曲线在续训点连续。

4. 解释 `loss.item()` 为何不会破坏计算图，而 `loss.numpy()` 会报错。
5. 设计一个指标记录格式（CSV 或 TensorBoard），包含学习率、训练损失、验证指标与耗时。

---

# 第 10 章 卷积神经网络

## 定义

**定义 10.1（二维离散卷积）** 对输入 $X \in \mathbb{R}^{C_{in} \times H \times W}$ 与卷积核 $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$，

$$Y[o,i,j] = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X[c,\, i \cdot s_h + u - p_h,\, j \cdot s_w + v - p_w] \cdot K[o,c,u,v] + b[o],$$

其中 $s$ 为步长（stride），$p$ 为填充（padding）。PyTorch 的 `Conv2d` 实现的是**互相关**（不翻转核），深度学习语境下通称卷积。

**定义 10.2（感受野）** 输出特征图上一个元素对应的输入区域大小。逐层累乘：$r_l = r_{l-1} + (k_l - 1)\prod_{m<l} s_m$。

**定义 10.3（批归一化 BatchNorm）** 训练时对每个通道统计批内均值与方差并归一化，同时以动量更新滑动统计量；评估时使用滑动统计量。

**定义 10.4（残差连接）** $y = x + \mathcal{F}(x)$，要求 $\mathcal{F}$ 的输出与 $x$ 形状一致；不一致时用 $1\times1$ 卷积投影。

## 性质与定理

**定理 10.1（输出尺寸公式）** 对输入边长 $H$、卷积核 $k$、填充 $p$、步长 $s$、膨胀率 $d$，

$$H_{out} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

等价的”same”填充（保持 $H_{out} = H$，$s = 1$）为 $p = \lfloor d(k-1)/2 \rfloor$。

**性质 10.1（参数量）** `Conv2d(C_in, C_out, k)` 的参数量为 $C_{out}(C_{in}k^2 + 1)$，**与输入分辨率无关**。这是卷积相对全连接的核心优势：全连接为 $C_{out}(C_{in}HW + 1)$。

**性质 10.2（计算量 FLOPs）** 单次前向的乘加次数约为 $H_{out}W_{out}C_{out}C_{in}k^2$，与分辨率成正比。

**性质 10.3（平移等变性）** 忽略边界效应时，卷积对输入平移具有等变性：$\mathcal{C}(\tau_\Delta X) = \tau_\Delta \mathcal{C}(X)$。池化提供局部平移不变性，但会损失定位精度。

**性质 10.4（BatchNorm 的批依赖）** 训练时输出依赖同批其他样本；批过小（如 $B < 8$）时统计噪声大，应改用 `GroupNorm` 或 `LayerNorm`。

## 代码实现

```
import torch
import torch.nn as nn

conv = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
x = torch.randn(8, 3, 224, 224)
y = conv(x)                                   # (8,64,224,224)

# 输出尺寸速算
def out_size(h, k, p=0, s=1, d=1):
    return (h + 2 * p - d * (k - 1) - 1) // s + 1
print(out_size(224, 3, 1, 1))                 # 224
print(out_size(224, 3, 1, 2))                 # 112

# 基本积木
bn = nn.BatchNorm2d(64)
pool = nn.MaxPool2d(2)
act = nn.ReLU(inplace=True)
```

```python
class BasicBlock(nn.Module):
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.short = nn.Identity()
        if stride != 1 or c_in != c_out:
            self.short = nn.Sequential(
                nn.Conv2d(c_in, c_out, 1, stride, bias=False),
                nn.BatchNorm2d(c_out))
    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return torch.relu(out + self.short(x))

class SmallResNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, 3, 1, 1, bias=False),
            nn.BatchNorm2d(32), nn.ReLU(inplace=True))
        self.layer1 = BasicBlock(32, 64, stride=2)
        self.layer2 = BasicBlock(64, 128, stride=2)
        self.head = nn.Linear(128, num_classes)
    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = x.mean(dim=(2, 3))               # 全局平均池化 -> (B,128)
        return self.head(x)
```

## 算例

**算例 10.1（参数量与 FLOPs 估算）** `Conv2d(64,128,3)`：参数 $128\times(64\times9+1)=73,856$；在 $56\times56$ 输入上 FLOPs 约 $56^2\times128\times64\times9\approx2.3\times10^8$。

**算例 10.2（感受野计算）** 三层 $3\times3$、步长 1：$r=1+(3-1)\times1=3$，再 5，再 7。即三个 $3\times3$ 等效一个 $7\times7$ 的感受野，但参数为 $3\times9=27$ 对 49，且多两次非线性。

**算例 10.3（`padding='same'` 的等价性）** 在 `stride=1`、`kernel_size` 为奇数时，`padding=k//2` 使输出与输入同尺寸。偶数核无法严格对称，需注意左右/上下各差一个像素。

## 练习

1. 推导 $k=5,p=2,s=2$ 时 $224\to$ 的输出尺寸。
2. 用 `unfold` 手动实现一次 $3\times3$ 卷积并与 `nn.Conv2d` 对比。
3. 解释为什么现代 CNN 多用 $3\times3$ 核而非大核。
4. 把 BatchNorm 换成 GroupNorm，比较小批量下的训练稳定性。
5. 计算 `BasicBlock(64,128,stride=2)` 的参数量。

---

# 第 11 章 序列模型：RNN、LSTM、GRU

## 定义

**定义 11.1（循环神经网络）** 对序列 $x_1, \dots, x_T$，隐状态递推

$$h_t = \tanh(W_{ih} x_t + b_{ih} + W_{hh} h_{t-1} + b_{hh}), \qquad y_t = W_{ho} h_t + b_o .$$

**定义 11.2（LSTM）** 引入细胞状态 $c_t$ 与三个门：

$$\begin{aligned} i_t &= \sigma(W_i [h_{t-1}, x_t] + b_i), \\ f_t &= \sigma(W_f [h_{t-1}, x_t] + b_f), \\ o_t &= \sigma(W_o [h_{t-1}, x_t] + b_o), \\ \tilde{c}_t &= \tanh(W_c [h_{t-1}, x_t] + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

**定义 11.3（GRU）** 用更新门 $z_t$ 与重置门 $r_t$ 合并 LSTM 的三个门，参数量约为 LSTM 的 3/4。

**定义 11.4（序列填充与掩码）** 变长序列需补齐到同一长度，并用 mask 标记有效位置，使损失与池化只在有效位置计算。

## 性质与定理

**性质 11.1（梯度消失与爆炸）** 反向传播沿时间展开，梯度含 $\prod_t (W_{hh}^{\top} \operatorname{diag}(1 - h_t^2))$。若谱半径 $< 1$ 则指数衰减，$> 1$ 则指数增长。LSTM 的加性细胞状态 $c_t = f_t \odot c_{t-1} + \dots$ 提供近似恒等的梯度通路，缓解消失问题。

**性质 11.2（`batch_first=True`）** PyTorch 默认输入形状为 `(T,B,F)`；指定 `batch_first=True` 后为 `(B,T,F)`。混用两种约定是最常见的形状错误来源。

**性质 11.3（`pack_padded_sequence`）** 对变长序列先排序、打包、过 RNN、再解包，可跳过填充位置的计算，结果与逐个处理等长序列一致。

**性质 11.4（双向与层数）** `bidirectional=True` 时输出维度翻倍；`num_layers>1` 时逐层堆叠，第二层起输入为上一层的隐状态序列。

## 代码实现

```
import torch
import torch.nn as nn
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

rnn = nn.LSTM(input_size=128, hidden_size=256, num_layers=2,
              batch_first=True, bidirectional=True, dropout=0.1)
x = torch.randn(16, 50, 128)          # (B,T,F)
out, (h_n, c_n) = rnn(x)
print(out.shape, h_n.shape)           # (16,50,512) (4,16,256)

# 变长序列
lengths = torch.tensor([50, 40, 30, 20])
packed = pack_padded_sequence(x[:4], lengths, batch_first=True, enforce_sorted=False)
out_p, (h_p, c_p) = rnn(packed)
out, _ = pad_packed_sequence(out_p, batch_first=True)   # 恢复 (4,50,512)
```

```python
# 掩码池化
mask = (torch.arange(50)[None, :] < lengths[:, None]).unsqueeze(-1)   # (4,50,1)
pooled = (out * mask).sum(1) / mask.sum(1)                            # (4,512)
```

## 算例

**算例 11.1（参数量）** `nn.LSTM(input=128, hidden=256)` 单向单层参数为 $4\times(128\times256+256\times256+2\times256)=394,240$。双向翻倍，两层再翻倍。

**算例 11.2（梯度裁剪的必要性）** 在字符级语言模型上，若不做裁剪，偶发的长序列会使梯度范数达到 $10^3$ 量级，一步更新即可破坏参数。常规做法是 `clip_grad_norm_(params, 1.0)`。

**算例 11.3（掩码求和的形状）** 若 `mask` 形状为 `(B,T)` 而 `out` 为 `(B,T,F)`，需 `mask.unsqueeze(-1)` 才能广播；否则维度对齐错误，得到 `(B,F,T)` 之类的结果。

## 练习

1. 手写单层 RNN 的前向，并与 `nn.RNN` 对比输出。
2. 解释 LSTM 中遗忘门偏置初始化为 1 的作用。
3. 用 `pack_padded_sequence` 处理变长序列，比较与直接 padding 的速度差异。
4. 在时间序列预测任务上比较 RNN、GRU、LSTM 的收敛速度。
5. 讨论为什么 Transformer 逐步取代 RNN 处理长序列。

---

# 第 12 章 注意力机制与 Transformer

## 定义

**定义 12.1（缩放点积注意力）** 给定查询 $Q \in \mathbb{R}^{n\times d_k}$、键 $K \in \mathbb{R}^{m\times d_k}$、值 $V \in \mathbb{R}^{m\times d_v}$，

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

批量形式为 `(B,h,n,d)` 与 `(B,h,m,d)`，输出 `(B,h,n,d)`。

**定义 12.2（多头注意力）** 将 $d$ 维拆成 $h$ 个头，每头独立做注意力后拼接，再经输出投影 $W_O$。

**定义 12.3（位置编码）** 由于注意力对序列顺序不敏感，需注入位置信息：正弦编码、可学习编码、或旋转位置编码（RoPE）。

**定义 12.4（Pre-LN 与 Post-LN）** Pre-LN 把归一化放在子层**之前**：$x \leftarrow x + \mathrm{Sublayer}(\mathrm{LN}(x))$；Post-LN 放在之后。Pre-LN 训练更稳定，是现代实现的主流。

## 性质与定理

**性质 12.1（缩放因子的来源）** 若 $q\cdot k$ 的各分量独立同分布、均值 0、方差 1，则其方差为 $d_k$。除以 $\sqrt{d_k}$ 使 logits 方差回到 1，避免 softmax 饱和导致梯度消失。

**性质 12.2（复杂度）** 自注意力的时间与显存复杂度均为 $O(n^2 d)$，长序列的主要瓶颈。FlashAttention 通过分块计算把显存降到 $O(n)$，数值上与原实现等价。

**性质 12.3（因果掩码）** 自回归解码时，对 $j > i$ 的位置置 $-\infty$ 后再 softmax，保证位置 $i$ 只能看到 $1..i$。

**性质 12.4（PyTorch 内置实现）** `torch.nn.functional.scaled_dot_product_attention` 在支持的设备上自动选择 Flash/Memory-Efficient/Math 后端，接口为 `(Q,K,V, attn_mask=None, dropout_p=0.0, is_causal=False)`，输入形状 `(B,h,n,d)`。

## 代码实现

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

def attention(q, k, v, mask=None):
    # q: (B,h,n,d) k: (B,h,m,d) v: (B,h,m,d)
    d = q.size(-1)
    scores = q @ k.transpose(-2, -1) / math.sqrt(d)     # (B,h,n,m)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))
    attn = scores.softmax(dim=-1)
    return attn @ v, attn

# 使用内置高效实现
out = F.scaled_dot_product_attention(q, k, v, is_causal=True)

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.d = n_heads, d_model // n_heads
```

```
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)
        self.drop = nn.Dropout(dropout)
    def forward(self, x, mask=None):
        B, N, C = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q, k, v = [t.view(B, N, self.h, self.d).transpose(1, 2) for t in (q, k, v)]
        o = F.scaled_dot_product_attention(q, k, v, attn_mask=mask,
                                           dropout_p=self.drop.p if self.training else 0.0)
        o = o.transpose(1, 2).reshape(B, N, C)
        return self.out(o)

class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads, dropout)
        self.ln2 = nn.LayerNorm(d_model)
        self.mlp = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.GELU(), nn.Dropout(dropout),
            nn.Linear(d_ff, d_model))
        self.drop = nn.Dropout(dropout)
    def forward(self, x, mask=None):
        x = x + self.drop(self.attn(self.ln1(x), mask))     # Pre-LN
        x = x + self.drop(self.mlp(self.ln2(x)))
        return x
```

## 算例

**算例 12.1（注意力矩阵的规模）** 序列长度 $n = 512$、头数 $h = 8$、批大小 $B = 16$，注意力矩阵为 $16 \times 8 \times 512 \times 512$ 个浮点数，float32 下约 134 MB。$n$ 翻倍则显存变 4 倍，这是长上下文的主要限制。

**算例 12.2（因果掩码的构造）**

```
n = 4
mask = torch.tril(torch.ones(n, n)).bool()   # 下三角为 True
```

`is_causal=True` 会在内部等价地构造该掩码。

**算例 12.3（参数量）** 一个 `TransformerBlock(d_model=512, n_heads=8, d_ff=2048)` 的参数量约为 $4d^2$（注意力）$+ 8d^2$（前馈）$= 12d^2 \approx 3.1 \times 10^6$。

## 练习

1. 手写多头注意力并用 `F.scaled_dot_product_attention` 校验数值一致。
2. 解释为什么 Pre-LN 在深层网络中更稳定。
3. 构造因果掩码，验证位置 $i$ 的输出不依赖 $j > i$ 的输入。
4. 比较正弦位置编码与可学习位置编码在长度外推上的差异。
5. 计算 `d_model=768, n_heads=12, n_layers=12` 的 Transformer 编码器参数量。

---

# 第 13 章 GPU、显存与性能

## 定义

**定义 13.1（主机与设备）** CPU 及其内存称为主机（host），GPU 及其显存称为设备（device）。二者通过 PCIe/NVLink 通信，带宽远低于显存内部带宽。

**定义 13.2（CUDA 流）** GPU 上的任务队列。同一流内任务顺序执行，不同流可并发。PyTorch 默认使用默认流。

**定义 13.3（同步点）** 使主机等待设备完成的调用，如 `.item()`、`.cpu()`、`torch.cuda.synchronize()`。计时必须在同步之后。

**定义 13.4（显存占用组成）** 参数、梯度、优化器状态、激活（前向中间结果）、临时缓冲区。Adam 的优化器状态为参数量的 2 倍（一阶与二阶矩）。

## 性质与定理

**性质 13.1（显存估算）** 以参数量 $P$、批大小 $B$ 计，训练显存近似为

$$M \approx 4P\,(\text{参数, fp32}) + 4P\,(\text{梯度}) + 8P\,(\text{Adam 状态}) + M_{\text{act}}(B).$$

混合精度下参数与梯度可降至 2 字节，但需保留 fp32 主权重。

**性质 13.2（异步执行）** 核函数启动是异步的，Python 侧计时若不 `synchronize()`，测到的是”下发耗时”而非”执行耗时”。

**性质 13.3（`torch.compile`）** 通过图捕获与算子融合减少 Python 开销与访存。首次调用有编译开销，适合长训练；对动态形状需允许重编译。

**性质 13.4（梯度检查点）** `torch.utils.checkpoint.checkpoint` 不保存中间激活，反向时重算。以约 1.33 倍计算量换取显存下降，适合深层网络。

## 代码实现

```python
import torch, time

dev = "cuda" if torch.cuda.is_available() else "cpu"

# 正确的计时
torch.cuda.synchronize()
t0 = time.perf_counter()
for _ in range(10):
    y = model(x)
torch.cuda.synchronize()
print((time.perf_counter() - t0) / 10)

# 显存
print(torch.cuda.memory_allocated() / 2**20, "MiB")
print(torch.cuda.max_memory_allocated() / 2**20, "MiB")
torch.cuda.reset_peak_memory_stats()

# 编译加速
compiled = torch.compile(model, mode="reduce-overhead")

# 梯度检查点
```

```python
from torch.utils.checkpoint import checkpoint
def forward_with_ckpt(self, x):
    for layer in self.layers:
        x = checkpoint(layer, x, use_reentrant=False)
    return x

# 自动选择设备与 AMP 见第 9 章
```

## 算例

**算例 13.1（显存上限决定批大小）** 模型 $P = 25 \times 10^6$，fp32 训练：参数 100 MB、梯度 100 MB、Adam 状态 200 MB，合计 400 MB；剩余显存用于激活。若 8 GB 卡留 1 GB 给系统，则激活预算约 6.6 GB，据此二分搜索最大批大小。

**算例 13.2（计时陷阱）** 不加 `synchronize()` 时，测得的”单步 2 ms”可能只是核函数下发时间，真实执行可能为 20 ms。对照方法：用 `torch.profiler` 查看设备侧耗时。

**算例 13.3（`num_workers` 与 GPU 利用率）** 若 `nvidia-smi` 显示 GPU 利用率在 20%–40% 波动，通常说明数据加载是瓶颈，应增大 `num_workers` 或启用 `pin_memory`。

## 练习

1. 用 `torch.profiler` 找出模型中耗时最长的三个算子。
2. 测量开启 `torch.compile` 前后的吞吐，并说明首次调用的编译开销。
3. 用梯度检查点把显存降到可容纳两倍批大小，测量吞吐变化。
4. 解释为什么 `.item()` 会强制同步，以及如何在训练循环中避免频繁调用。
5. 估算你在第 10 章 `SmallResNet` 上能用的最大批大小。

---

# 第 14 章 保存、加载、导出与部署

## 定义

**定义 14.1（序列化）** 把 Python 对象转为字节流的过程。`torch.save` 使用 pickle 协议，可保存张量、字典、模块对象。

**定义 14.2（`weights_only`）** `torch.load` 的参数。PyTorch 2.6 起默认 `True`，只允许加载安全的基础类型（张量、基本容器）；加载包含任意 Python 对象（如整个 `nn.Module` 实例）的旧检查点时会抛出 `UnpicklingError`。

**定义 14.3（TorchScript）** 把模型转为与 Python 解耦的中间表示，可在无 Python 环境运行。两种获得方式：`torch.jit.trace`（记录实际执行的算子，不支持数据依赖的控制流）与 `torch.jit.script`（解析源码，支持控制流）。

**定义 14.4（ONNX）** 开放的跨框架模型交换格式。PyTorch 通过 `torch.onnx.export` 导出，可在 ONNX Runtime、TensorRT 等后端执行。

**定义 14.5（量化）** 以低比特整数（int8）表示权重/激活，减小模型体积并提升推理速度。分为训练后量化（PTQ）与量化感知训练（QAT）。

## 性质与定理

**性质 14.1（推荐保存内容）** 保存 `state_dict()` 而非整个模型。前者与代码结构解耦、体积小、可跨版本加载；后者依赖类的定义路径，重构代码后可能无法加载。

**性质 14.2（`map_location` 的作用）** 在无 GPU 的机器上加载 GPU 训练的检查点，必须指定 `map_location="cpu"`，否则反序列化时会尝试分配 CUDA 张量并报错。

**性质 14.3（`strict=False` 的部分加载）** `load_state_dict(..., strict=False)` 允许键不匹配，返回 `missing_keys` 与 `unexpected_keys`。用于加载骨干网络权重或迁移学习；生产环境应检查返回值而非忽略。

**性质 14.4（trace 的局限）** `torch.jit.trace` 只记录一次前向所经过的路径，`if` 分支与循环次数被固化。含有数据依赖控制流的模型必须用 `script`。

**性质 14.5（量化的一致性问题）** PTQ 需要校准数据统计激活范围；校准集分布偏离推理分布时精度下降明显。QAT 在训练中模拟量化误差，精度更接近浮点模型。

## 代码实现

```
import torch
import torch.nn as nn

# 1) 仅权重(推荐)
torch.save(model.state_dict(), "weights.pt")
sd = torch.load("weights.pt", map_location="cpu", weights_only=True)
model.load_state_dict(sd)

# 2) 完整检查点(含优化器,用于续训)
torch.save({"model": model.state_dict(),
            "optimizer": opt.state_dict(),
            "epoch": epoch}, "ckpt.pt")
ckpt = torch.load("ckpt.pt", map_location="cpu", weights_only=False)
```

```python
# 3) 加载旧版整对象检查点
old = torch.load("legacy.pth", map_location="cpu", weights_only=False)

# 4) TorchScript
model.eval()
traced = torch.jit.trace(model, torch.randn(1, 3, 224, 224))
traced.save("model_traced.pt")
traced = torch.jit.load("model_traced.pt")

scripted = torch.jit.script(model)          # 支持控制流

# 5) ONNX
torch.onnx.export(model, torch.randn(1, 3, 224, 224), "model.onnx",
                  input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
                  opset_version=17)

# 6) 训练后量化(int8)
qmodel = torch.ao.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8)

# 7) 新式导出(torch.export,推荐用于部署)
ep = torch.export.export(model, (torch.randn(1, 3, 224, 224),))
torch.export.save(ep, "model.pt2")
```

## 算例

**算例 14.1（跨设备加载）** 训练在 `cuda:0`，推理在 CPU：`torch.load(path, map_location="cpu")`。若省略该参数，反序列化会尝试恢复 CUDA 张量，在无 CUDA 环境下失败。

**算例 14.2（迁移学习的部分加载）** 把 ImageNet 预训练的 ResNet 主干载入自建模型：

```python
sd = torch.load("resnet18.pth", map_location="cpu", weights_only=True)
own = model.state_dict()
own.update({k: v for k, v in sd.items() if k in own and v.shape == own[k].shape})
model.load_state_dict(own)
```

该写法显式保证形状一致，比 `strict=False` 更可控。

**算例 14.3（量化后的体积与速度）** 以 $10^7$ 参数的 MLP 为例，fp32 约 40 MB，int8 约 10 MB；在 CPU 推理上通常加速 2–4 倍，GPU 上收益取决于是否支持 INT8 Tensor Core。

## 练习

1. 分别保存 `state_dict` 与整个模型，比较文件大小并说明差异原因。
2. 构造一个含 `if` 分支的模型，验证 `trace` 与 `script` 的行为差异。
3. 导出一个 ONNX 模型并用 `onnxruntime` 加载推理，对比与 PyTorch 输出的数值误差。
4. 对线性层做动态量化，测量体积与延迟变化。
5. 解释 `weights_only=True` 为什么能提升安全性。

---

# 第 15 章 调试、可复现性与常见错误

## 定义

**定义 15.1（可复现性）** 相同代码与相同随机种子在相同环境下产生相同结果。完全确定还需控制 CUDA 算子选择、并行归约顺序与数据加载顺序。

**定义 15.2（数值不稳定性）** 因浮点舍入、上溢/下溢或相减抵消导致的误差放大，表现为 `nan`、`inf` 或训练震荡。

**定义 15.3（过拟合与欠拟合）** 训练损失低而验证损失高为过拟合；两者都高为欠拟合。

## 性质与定理

**性质 15.1（随机源清单）** 需要固定的随机源包括：Python `random`、NumPy、`torch`、CUDA（`torch.cuda.manual_seed_all`）、DataLoader 的 worker 种子、cuDNN 的算法选择（`torch.backends.cudnn.deterministic=True`）。此外 `torch.use_deterministic_algorithms(True)` 可强制确定性，代价是部分算子不可用或变慢。

**性质 15.2（`nan` 的产生路径）** 常见来源：① 学习率过大导致参数发散；② 对数/除法作用于 0；③ `softmax` 输入含 `inf`；④ 半精度下梯度下溢；⑤ 自定义 `backward` 有误。定位手段是逐层检查前向输出是否有限。

**性质 15.3（形状错误的定位顺序）** 先打印每层输入输出形状，再核对批维与特征维的位置；重点检查 `view/reshape` 前后、`cat` 的 `dim`、以及广播是否意外发生（可用 `torch.broadcast_shapes` 预判）。

**性质 15.4（显存泄漏的常见来源）** 在训练循环中累积带梯度的张量（如把 `loss` 存进列表）、忘记 `no_grad` 的评估、以及持有计算图引用的全局缓存。

## 代码实现

```
import os, random
import numpy as np
import torch

def set_seed(seed=42, deterministic=True):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    if deterministic:
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False
        torch.use_deterministic_algorithms(True, warn_only=True)

# 逐层形状检查
from torch import nn
def shape_trace(model, x):
    hooks = []
    def mk(name):
        def h(m, i, o):
            print(f"{name:30s} {tuple(i[0].shape)} -> {tuple(o.shape)}")
        return h
    for n, m in model.named_modules():
        if len(list(m.children())) == 0:
```

```
            hooks.append(m.register_forward_hook(mk(n)))
    model(x)
    for h in hooks:
        h.remove()

# NaN 定位
torch.autograd.set_detect_anomaly(True)      # 反向出 nan 时定位到具体算子
```

## 算例

**算例 15.1（loss 变 nan 的排查顺序）** ① 把学习率降 10 倍重跑；② 检查损失函数是否对 0 取对数；③ 用 `torch.isnan(x).any()` 逐层定位；④ 若启用 AMP，先关闭 AMP 复现；⑤ 检查输入是否含 `inf` 或异常大值。

**算例 15.2（训练/验证指标差异过大的诊断）** 若训练准确率 99%、验证 60%，先确认验证是否在 `eval()` 与 `no_grad()` 下进行；再检查训练集与验证集是否有重叠；最后考虑正则化与数据增强强度。

**算例 15.3（同种子仍不可复现）** 原因通常是 DataLoader 的 `num_workers>0` 未设置 worker 种子，或使用了非确定性的 CUDA 算子（如 `atomicAdd` 归约）。设置 `worker_init_fn` 与 `generator` 可解决前者。

## 练习

1. 写一个 `set_seed` 并在同一脚本中跑两次，验证结果完全一致。
2. 人为把学习率设为 10，观察并记录 `nan` 出现的步数。
3. 用 `shape_trace` 检查第 10 章 `SmallResNet` 每层形状。
4. 构造一个显存持续增长的训练循环，并用 `torch.cuda.memory_allocated()` 定位。
5. 讨论 `torch.use_deterministic_algorithms(True)` 对性能的影响。

---

# 第 16 章 综合实战：MNIST → CIFAR-10 → 小型 NeRF

本章不引入新概念，而是把前 15 章的内容组织为三个递进的完整任务。每个任务给出目标、数据、模型、训练配置与验收标准。

## 任务一：MNIST 手写数字分类

**目标** 掌握完整的训练—评估—保存流程，验证环境正确。

- 数据：$28 \times 28$ 灰度，10 类，60000 训练 / 10000 测试。
- 模型：两层 MLP（$784 \to 256 \to 10$）或小型 CNN（两个 $3 \times 3$ 卷积 + 全连接）。
- 配置：`batch_size=64`、`Adam(lr=1e-3)`、10 个 epoch、`CrossEntropyLoss`。
- 验收：测试准确率 $\geq 98\%$（MLP）或 $\geq 99\%$（CNN）；训练损失单调下降。
- 观察点：`model.train()/eval()` 的切换、`no_grad` 评估、检查点保存与恢复。

## 任务二：CIFAR-10 图像分类

**目标** 掌握卷积网络、数据增强、学习率调度与正则化。

- 数据：$32 \times 32$ 彩色，10 类，50000 / 10000。
- 模型：第 10 章的 `SmallResNet` 或 ResNet-18。
- 增强：`RandomCrop(32, padding=4)`、`RandomHorizontalFlip`、`Normalize`。
- 配置：`batch_size=128`、`SGD(momentum=0.9, weight_decay=5e-4)` 或 `AdamW`、`CosineAnnealingLR`、50 个 epoch、AMP。
- 验收：测试准确率 $\geq 90\%$（ResNet-18）或 $\geq 85\%$（SmallResNet）。
- 观察点：学习率曲线与验证准确率的对应关系；关闭数据增强后的过拟合程度；AMP 对显存与吞吐的影响。

## 任务三：小型神经辐射场

**目标** 理解”渲染即张量运算”，把神经网络用于连续场表示。

**问题设定** 用 MLP 表示一个三维场景：输入空间坐标 $(x, y, z)$ 与视线方向 $(\theta, \phi)$，输出体积密度 $\sigma$ 与颜色 $c$。沿光线采样 $N$ 个点，按体渲染公式积分得到像素颜色：

$$\hat{C}(r) = \sum_{i=1}^{N} T_i (1 - e^{-\sigma_i \delta_i}) c_i, \qquad T_i = \exp\Big(-\sum_{j<i} \sigma_j \delta_j\Big).$$

**实现要点**

1. 位置编码：把坐标映射到高频正弦基，提升对细节的表示能力；
2. 分层采样：粗网络均匀采样，细网络在密度高的区间加密采样；
3. 体渲染积分：上式对 `(B, N)` 的张量运算，注意 `cumsum` 与 `exp` 的数值范围；
4. 损失：渲染颜色与真值像素的均方误差；
5. 形状约定：光线批 `(B, N, 3)`，密度 `(B, N, 1)`，颜色 `(B, N, 3)`，最终像素 `(B, 3)`。

**验收** 在单场景（如合成球体或小房间）上训练至 PSNR $\geq 25$ dB；渲染图与真值视觉一致。

**与本仓库的关联** 第 12 章的注意力与本章的体渲染同属”对一组元素加权求和”的范式：注意力按相关性加权，体渲染按透射率加权。CBF+NeRF 项目正是在此表示之上，用渲染深度构造观测型控制屏障函数。

## 训练配置对照表

| 任务 | 数据规模 | 模型参数量 | 批大小 | 优化器 | 预期耗时（单卡） |
|---|---|---|---|---|---|
| MNIST | 6 万 | $\sim 2\times10^5$ | 64 | Adam 1e-3 | 数分钟 |
| CIFAR-10 | 5 万 | $\sim 1.1\times10^7$ | 128 | SGD 0.1 | 1–2 小时 |
| 小型 NeRF | 单场景 | $\sim 10^6$ | 1024 光线 | Adam 5e-4 | 数十分钟 |

## 练习

1. 完成三个任务并记录每个任务的损失曲线与验收指标。
2. 在任务二上比较”有/无数据增强”的验证准确率。
3. 在任务三上把位置编码的最高频率提高一倍，观察 PSNR 变化。
4. 给任务三加入混合精度，检查是否出现 `nan` 并说明原因。
5. 把任务三的体渲染改为”按注意力加权”，讨论与标准体渲染的等价条件。

---

# 附录 A API 速查表

| 类别 | 常用接口 | 说明 |
| --- | --- | --- |
| 构造 | `torch.tensor` `zeros` `ones` `full` `empty` `eye` `arange` `linspace` | 工厂函数优先于 `torch.Tensor` |
| 随机 | `rand` `randn` `randint` `randperm` `manual_seed` | 可复现需固定种子 |
| 形状 | `view` `reshape` `permute` `transpose` `squeeze` `unsqueeze` `expand` `repeat` `contiguous` | 前六个通常零拷贝 |
| 合并 | `cat` `stack` `split` `chunk` `flatten` | `cat` 不增维，`stack` 增维 |
| 归约 | `sum` `mean` `max` `min` `argmax` `norm` | `max(dim=)` 返回 `(values,indices)` |
| 线代 | `matmul` `mm` `bmm` `einsum` `@` | 批量用 `matmul` |
| 自动微分 | `requires_grad_` `backward` `autograd.grad` `detach` `no_grad` `inference_mode` | 叶张量才有 `.grad` |
| 模块 | `nn.Module` `nn.Parameter` `register_buffer` `state_dict` `load_state_dict` `apply` | 列表容器会漏注册 |
| 层 | `Linear` `Conv2d` `BatchNorm2d` `LayerNorm` `Dropout` `Embedding` | 注意 `bias` 与归一化顺序 |
| 损失 | `CrossEntropyLoss` `BCEWithLogitsLoss` `MSELoss` `L1Loss` | CE 接受 logits |
| 优化 | `SGD` `Adam` `AdamW` `zero_grad` `step` | AdamW 解耦权重衰减 |
| 调度 | `StepLR` `CosineAnnealingLR` `OneCycleLR` `ReduceLROnPlateau` | 调用频率须与 `T_max` 一致 |
| 数据 | `Dataset` `DataLoader` `Sampler` `collate_fn` | `num_workers` 与 `pin_memory` |
| 保存 | `save` `load` `jit.trace` `jit.script` `onnx.export` `export.export` | `weights_only` 默认 `True` |
| 性能 | `autocast` `GradScaler` `compile` `checkpoint` `profiler` | 先测同步后的时间 |

| 类别 | 常用接口 | 说明 |
| --- | --- | --- |
| 设备 | `device` `to` `cuda.is_available` `memory_allocated` | 同运算须同设备 |

# 附录 B 练习参考答案要点

**第 1 章** ① `x[1]`、`x.diagonal()`、`x.sum(dim=0)`；② `view` 与原张量共享 `storage`，修改其一另一可见；③ 前者在 CPU 构造后再拷贝，后者直接在设备上分配；④ `permute(0,2,3,1)`，后续 `view` 前需 `contiguous`；⑤ `float32`，类型提升规则向更高精度对齐。

**第 2 章** ① 均可行，结果分别为 `(5,6)`、报错、`(2,4,3)`、标量；② `x.permute(0,2,1)` 或 `einsum('bnc->bcn')`；③ `x.flatten(2).norm(dim=-1)`；④ `+=` 是原地写，破坏 autograd 记录；⑤ 减去最大值后 `exp` 求和再取对数。

**第 3 章** ① $\partial f/\partial x_i = \mathrm{softmax}(x)_i$；② `x += 1` 触发 in-place 错误；③ $\nabla = (A + A^\top)x$；④ `no_grad` 影响后续所有运算，`detach` 只影响该张量；⑤ $x = 0$ 处子梯度取 0 或 1 均可，需在文档中声明。

**第 4 章** ① 增加二次项只需改 `pred` 表达式；② 用列表记录 `lr` 并在 epoch 末修改 `optimizer.param_groups[0]['lr']`；③ `torch.autograd.grad` 返回梯度但不写入 `.grad`；④ 对 SGD 等价，对 Adam 不等价；⑤ loss 曲线单调上升或剧烈震荡。

**第 5 章** ① `named_parameters()` 返回名与张量；② `Sequential` 用于线性串联，`ModuleList` 用于需要自定义前向的结构；③ buffer 出现在 `state_dict` 但不在 `parameters`；④ hook 返回的 `out` 需 `detach`；⑤ 冻结后 `grad` 为 `None`。

**第 6 章** ① `RandomCrop` + `ToTensor`；② 记录两个 epoch 的实际耗时；③ padding 值通常取 0 并配合 mask；④ `WeightedRandomSampler` 以逆频率为权重；⑤ 多进程下状态不共享，写入会丢失。

**第 7 章** ① `log_softmax` 后取 `nll_loss`；② 比较召回率与精确率；③ 偏差修正消除初始零初始化带来的偏置；④ 记录三条验证曲线；⑤ 直接置 `None` 释放引用。

**第 8 章** ① 绘制两条验证曲线；② `LambdaLR` 接收 `epoch` 返回缩放系数；③ 按 `p.ndim` 与名称分组；④ 宽网络通常需要更大 $p$；⑤ 两种写法数值应一致到 $10^{-7}$。

**第 9 章** ① 类中维护 `best` 与 `patience`；② 用 `max_memory_allocated` 与计时；③ 比较续训点前后的 loss；④ `item()` 取值不建图，`numpy()` 要求无梯度；⑤ CSV 每行一个 epoch。

**第 10 章** ① $\lfloor (224 + 4 - 4 - 1)/2 \rfloor + 1 = 112$；② `unfold` 后矩阵乘；③ 更少参数、更多非线性；④ GroupNorm 在小批量下更稳定；⑤ $128 \times (64 \times 9 + 1) + 128 \times (128 \times 9 + 1) + 128 \times (64 + 128)$ 量级。

**第 11 章** ① 按递推式写循环；② 初始遗忘门接近 1 有利于保留信息；③ 变长时跳过填充计算；④ 比较收敛 epoch；⑤ 并行性与长程依赖。

**第 12 章** ① 两者数值一致到 $10^{-6}$；② 梯度不随深度衰减；③ 掩码保证因果性；④ 可学习编码难以外推到训练长度之外；⑤ $12 \times 768^2 \times 12 \approx 8.5 \times 10^7$。

**第 13 章** ① `profiler` 按 `cuda_time_total` 排序；② 记录编译开销与稳态吞吐；③ 显存与吞吐权衡；④ `.item()` 触发同步；⑤ 依据显存公式二分。

**第 14 章** ① `state_dict` 体积约为参数量乘以元素字节数；② `trace` 固化分支；③ 误差应在 $10^{-5}$ 量级；④ 体积约降为 1/4；⑤ 禁止反序列化任意代码。

**第 15 章** ① 两次运行的损失逐位相同；② 记录首次出现 `nan` 的步数；③ 每层形状应与设计一致；④ `memory_allocated` 持续上升说明有引用未释放；⑤ 确定性算法通常更慢。

# 附录 C 官方资源索引

| 资源 | 地址 | 用途 |
| --- | --- | --- |
| PyTorch 文档 | https://docs.pytorch.org/docs/stable/index.html | API 权威定义 |
| Learn the Basics | https://docs.pytorch.org/tutorials/beginner/basics/intro.html | 官方入门七步 |
| Deep Learning with PyTorch: 60 Minute Blitz | https://docs.pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html | 快速通览 |
| Learning PyTorch with Examples | https://docs.pytorch.org/tutorials/beginner/pytorch_with_examples.html | 同一模型的多种写法 |
| What is torch.nn really? | https://docs.pytorch.org/tutorials/beginner/nn_tutorial.html | 从手写到 `nn` 的过渡 |
| Understanding requires_grad / leaf tensors | https://docs.pytorch.org/tutorials/beginner/understanding_leaf_vs_nonleaf_tutorial.html | 自动微分细节点 |
| Performance Tuning Guide | https://docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html | 性能优化清单 |
| Automatic Mixed Precision | https://docs.pytorch.org/tutorials/recipes/recipes/amp_recipe.html | AMP 实践 |
| torch.compile 入门 | https://docs.pytorch.org/tutorials/intermediate/torch_compile_tutorial.html | 图编译 |
| DDP 入门 | https://docs.pytorch.org/tutorials/intermediate/ddp_tutorial.html | 多卡训练 |
| ONNX 导出 | https://docs.pytorch.org/tutorials/beginner/onnx/intro_onnx.html | 部署 |
| Profiler | https://docs.pytorch.org/tutorials/recipes/recipes/profiler_recipe.html | 性能剖析 |

## 版本与勘误说明

- 本讲义以 PyTorch 2.x 为基准；`torch.load` 的 `weights_only` 默认值在 2.6 变更，`torch.jit` 系列在新版本中逐步让位于 `torch.export`。
- 所有接口名与参数名均以官方文档为准；若发现与本文不一致，以官方文档为最终依据。